EXAONE Demand 1.0:
A Time Series Foundation Model for Demand Forecasting

LG AI Research
Wonbin Ahn, Hwanil Choi, Sangjun Han, Dongwan Kang, Junhyeok Kang, Minjae Kim,
Jaehoon Lee, Seunghan Lee, Soonyoung Lee, Tae Yoon Lim, Jun Seo, Sungdong Yoo

Time series foundation models (TSFMs) are pretrained on series from diverse domains, where demand series make up only a small fraction. Demand data has properties that such corpora rarely contain: Short histories, frequent zeros, censoring by stock-outs, and exogenous events that the series does not record. To this end, we propose **EXAONE Demand**, built on 1) a *demand-specific corpus* and 2) a *demand-aware adapter*. For the corpus, we assemble 11.3M series and 48.4B observations from 73 sources, and a *synthetic generator* supplies the behaviour that open demand data under-represents. For the adapter, we attach low-rank branches to a frozen general-domain backbone, one for each of the four demand classes (smooth, intermittent, erratic, and lumpy), and a router that reads eight scale-free statistics of the input series decides how much each branch contributes. We build EXAONE Demand in two versions, one trained on real-world and synthetic demand together and one trained on the synthetic corpus alone. On 22 held-out datasets, both versions outperform 36 TSFMs, and real-world demand adds a gain over synthetic data alone.

Model `https://huggingface.co/LG-AI-Research/EXAONE-Demand-1.0`

Code `https://github.com/LGAI-Research/EXAONE-Forecast`

1 Introduction

Demand forecasting predicts how much of a product, a service, or a resource will be requested over a future period, and that prediction decides what an organisation stocks, staffs, and ships [51]. Its two directions of error carry different costs, where *under-forecasting* leaves demand unmet and *over-forecasting* commits resources that are never used. Demand behaves unlike the *smooth* and *long* series that dominate general forecasting benchmarks, and it is therefore studied as a problem of its own, with its own error measures and its own taxonomy of series [66].

Time series foundation models (TSFMs) are pretrained at scale to forecast across domains [4, 17, 26, 72], and they reach strong zero-shot accuracy on general benchmarks [2, 25]. Their corpora are assembled for *breadth* rather than for *demand*, and the properties that make demand hard are diluted by the far more numerous other series. We argue that *demand forecasting needs a corpus of its own*, one that reflects these properties instead of averaging them out.

Three properties of demand data. Three properties recur across our demand series and are rare in general corpora, and Figure 1 shows all three in eight series, one drawn from each of eight open sources of the corpus.

- **Intermittency.** A large share of series are *zero* most of the time, broken by irregular non-zero periods, which the *average demand interval* (ADI) and the *squared coefficient of variation* (CV^2) characterise [66].
- **Shortness.** Demand is recorded at the granularity at which it is *ordered*, which is weekly or monthly per SKU and per store, leaving individual series far shorter than the ones general benchmarks are built on.
- **Exogenous and censored structure.** Promotions, holidays, and life-cycles drive level shifts whose cause is *absent from the series*, and observed sales are a *censored* view of latent demand whenever inventory runs out.

A demand corpus does not hold one kind of series but four, where *smooth*, *intermittent*, *erratic*, and *lumpy* demand arrive mixed together. They ask different questions of a forecaster, where *intermittency* turns the question into *when* demand arrives, while *erratic* and *lumpy* behaviour turns it into *how much* arrives at once. A corpus that holds all four is not enough by itself, since one set of weights fitted to the mixture has to serve both questions, and moving it toward one behaviour costs accuracy on the other, a trade-off that a single adapter has no means to resolve.

Authors are listed in alphabetical order by last name. Corresponding author: Wonbin Ahn (wonbin.ahn@lgresearch.ai)

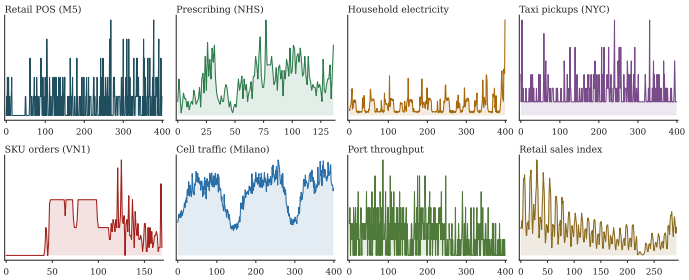


Figure 1: **Representative real-world demand series.** One series is drawn per source from the corpus. Retail and SKU-level series are dominated by zeros and irregular spikes, while energy and transport series are long with strong daily and weekly cycles.

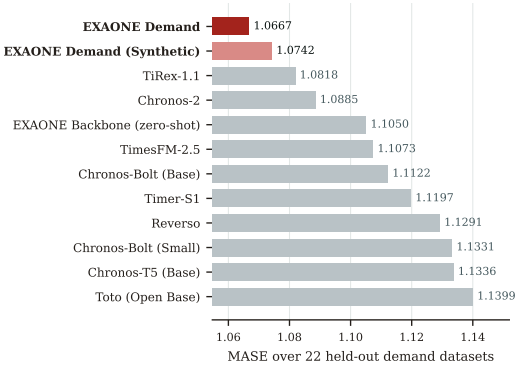


Figure 2: Performance on demand datasets.

Table 1: **Collected sources by domain.** The count is over the 93 collected sources, and the last two rows are source types rather than domains, since each of them holds many datasets under a single published name.

Domain	Sources	Representative content
Retail and e-commerce	24	Store and SKU sales, grocery baskets, fashion and online retail
Energy	23	Household and industrial electricity load, national demand
Transport	14	Taxi and ride-hailing pickups, bike share, transit ridership, port calls
Healthcare	6	Prescription dispensing at practice, pharmacy, and hospital level
Service and hospitality	4	Restaurant covers, hotel and booking demand
Telecommunications	3	Cell-grid traffic
Economics	3	Retail sales indices, industrial production, trade volumes
Web and compute	2	Page requests, virtual-machine resource usage
General-purpose corpora	7	Time-300B, GIFT-Eval, LOTSA, Chronos, fev-bench, Monash
Competition benchmarks	7	M4, M5, VN1, Rossmann, Favorita, and related

To this end, we propose **EXAONE Demand**, a TSFM for demand forecasting that *specialises by demand behaviour*. As Figure 3 lays out, it rests on two components: 1) A *demand-specific corpus* built from real-world demand sources and a synthetic generator, and 2) a *demand-aware adapter* that mixes low-rank branches by the behaviour of the input series. Specifically, we adapt a general-domain TSFM pretrained on KernelSynth [4] alone, namely synthetic series drawn from Gaussian process kernels, keeping its weights frozen and training only five low-rank branches, one shared by every series and four tied to the demand classes, where a *router* decides how much each of the four contributes. We build EXAONE Demand in two versions, one trained on real-world and synthetic demand together and one trained on the synthetic corpus alone. As shown in Figure 2, both versions outperform the state-of-the-art (SoTA) TSFMs on 22 held-out demand datasets, with real-world demand adding a gain over synthetic data.

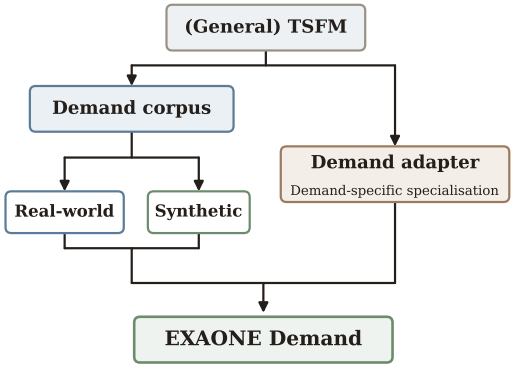


Figure 3: Overview of EXAONE Demand.

2 Construction of Demand Dataset

We construct the demand dataset in the following three steps, as illustrated in Figure 4:

- **Collection** (§2.1): We gather various sources and keep only the series that measure demand.
- **Preprocessing** (§2.2): We convert every source into a common schema and index every series we keep.
- **Train-test split** (§2.3): We assign sources by their published role and remove the training series that leak.

A synthetic corpus is built in parallel, and the real and synthetic corpora stay apart until the last step, where every synthetic series is used for training. Section 3 describes the design and the generating process behind it.

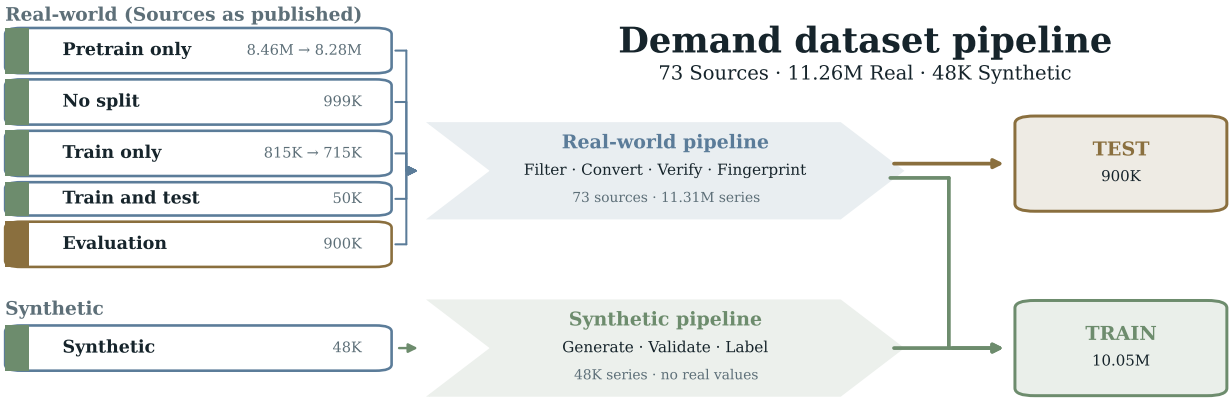


Figure 4: **Pipeline for demand dataset construction. 1) Collection:** Real sources are grouped by their published role, and the synthetic corpus forms a group of its own below. **2) Preprocessing:** The two corpora run on separate tracks. The real track filters, converts, and verifies each source before removing leaked series, while the synthetic track generates, validates, and labels its own. **3) Train-test split:** Of the five roles, only the evaluation suites reach the test side, and the other four are used for training, together with every one of the synthetic series.

2.1 Collection

The first decision is not *where* to find data but *what* to accept as demand. This cannot be taken from the data, as nothing in a published file marks a series as demand and no metadata field records it. We therefore state a definition of our own, taking a demand series to be a measurement of *consumption, sales, or utilisation per unit of time by an identifiable entity*. What the definition rejects matters as much as what it keeps, since the rejected series are often the ones that look most like demand. Three rejections came up in almost every source, and we state them as rules:

- ***1) Supply is not demand.*** We keep a series only when it measures what was taken, not what was produced. Solar and wind generation are therefore excluded, and so is the `export` reading of a residential smart meter, while the `import` reading of the same meter is kept, since it records what the household actually drew.
- ***2) Physical quantities are not demand.*** Transformer oil temperature, air quality, river flow, and weather measure the state of a system rather than anyone's use of it. Several of them are standard forecasting benchmarks, and they therefore reach us inside the broad collections described below rather than as sources of their own.
- ***3) Inventory is not demand.*** The number of bikes available at a docking station is supply, and the number of trips starting there is demand. The same file often holds both, and we take only the second.

Two cases are not decided by the rules above, since the question there is not whether a series is demand but what a single number in it stands for. (1) **Sources with no series:** Event logs such as taxi trips, bike rentals, and municipal service requests store one row per event. They become demand only after the events are counted per entity and per time step, which makes those two settings the definition of demand for that source. (2) **Sources with several candidate values:** In pharmaceutical data we take prescription *items* rather than *quantity*, since quantity is measured in tablets for one drug and in millilitres for another, and the two cannot be summed on a common scale.

Sources. Demand data is published in three types of sources, and relying on a single type yields a biased corpus, as benchmark suites and public data archives provide almost disjoint material. We therefore collect from all three:

- **Type 1) Forecasting benchmarks and competitions.** M4, M5, VN1, Rossmann, Favorita, and related retail competitions, which define the conventional evaluation of demand forecasting in the literature.
- **Type 2) General-purpose corpora.** Broad collections assembled by other groups to cover time series in general, from which we keep only the subsets that are relevant to demand and discard the rest.
- **Type 3) Domain-specific open data.** National health services, transport authorities, energy regulators, and statistical agencies, which publish demand directly but are absent from forecasting benchmarks.

We collect 93 sources in total, of which 73 are converted to a common schema. The remaining 20 are excluded because 1) their contents are not tabular, 2) they duplicate sources already collected, or 3) they require credentials we do not hold. Table 1 summarises the collection by domain, and Figure 6 shows the distribution of the converted corpus.

Published roles. Each source is released with a role determined by its publisher, which we record rather than assign. We identify five roles: 1) *Pretrain only*, corpora with no held-out part, 2) *No split*, releases published as a single

Table 2: **Collected sources by published role.** The role is a property of the release rather than a decision of ours, and what a source finally contributes is settled by the leakage check rather than by the role alone.

Published role	Sources	Series	Sources in this group
Pretrain only	8	8,542,433	Time-300B [63], GIFT-Eval pretrain [2], Chronos [4], LOTSA [72], EIA-930 [68], IMF PortWatch [32], NYC TLC [53], NYC 311 [56]
No split	67	999,102	Goiener smart meters [28], UCI electricity load [67], Citi Bike [12], Divvy [19], NHS dispensing [54], Online Retail II [9], OPSD [57], AEMO [6], and 59 others
Train only (Held-out private)	3	815,098	Web Traffic [35], Instacart [64], Kaggle sales forecasting [59]
Train and test	11	50,489	M4 [50], M5 [51], Favorita [36], VN1 [69], Rossmann [34], Walmart sales forecast [1], Walmart recruiting [33], Store Item Demand [37], Panama load (two releases) [49], Food demand [38]
Evaluation	3	899,618	GIFT-Eval [2], fev-bench [62], BOOM [18]

undivided file, 3) *Train only*, releases that publish a training file while keeping the held-out part private, 4) *Train and test*, releases that ship a fixed train and test pair, and 5) *Evaluation*, suites published to be scored against. Table 2 lists the sources in each role, and §2.3 describes how each role is used in training and in evaluation.

2.2 Preprocessing

Preprocessing brings sources of all three types into a single schema. The demand filter of §2.1 applies to all of them, but at a different level in each. For competitions and domain-specific open data (types 1 and 3) the source as a whole is demand, and the filter only decides which columns to take. A general-purpose corpus (type 2) holds demand and non-demand datasets side by side, and there the filter removes whole datasets. Of the seven general-purpose corpora in Table 1, six are converted, since Monash is already contained in the Chronos and LOTSA collections, and the filter keeps 269 of the 343 datasets they contain and drops the remaining 74. This large drop rate is expected, as these corpora are assembled to cover *time series* rather than *demand*, and the dropped datasets are mostly weather, generation, and air quality, which rules 1) and 2) of the demand filter in §2.1 already exclude.

General-purpose corpora. These six are not single datasets but collections, each one containing many datasets under a single name, and the corpus itself therefore has no single domain. Four were published for pretraining (Time-300B [63], the GIFT-Eval pretraining split [2], the Chronos training collection [4], LOTSA [72]) and two as evaluation suites (fev-bench [62], GIFT-Eval [2]). We therefore assign a domain to each dataset inside them rather than to the corpus as a whole. These six hold 83% of all series, and this one step therefore fixes the domain of most of the corpus.

Conversion and verification. All sources are converted to a single schema of `(series_id, timestamp, target)`, together with a per-series index recording the shape, demand character, and provenance of each series. The converted counts are then verified against numbers established *outside* the pipeline, such as competition documentation or the published number of entities, rather than against the conversion log itself. Appendix C records the schema in full and the four quantities that every source is required to agree with before it is admitted to the corpus.

Corpus statistics. The converted corpus contains 11.3M series and 48.4B observations from 73 sources. Aggregate size alone does not indicate whether a corpus is suited to demand, as a small number of long high-frequency series can dominate any total. Table 3 therefore reports the *shape* of the corpus instead, namely how long its series are, how they split across the four demand classes, and what share of their values is zero at each percentile.

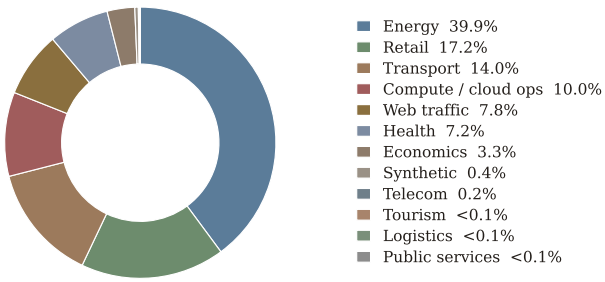


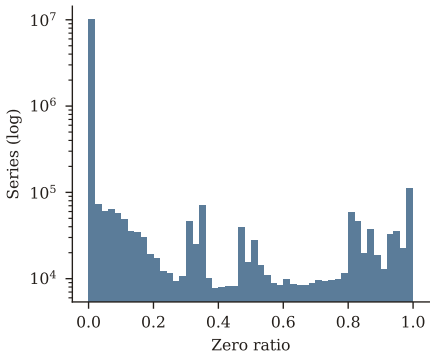
Figure 6: **Corpus composition by series count.** Every series carries one domain of the converted corpus.

Two characteristics of the corpus affect all downstream design decisions. First, the length distribution is heavily skewed, where the median series has 1,373 points while the lower quartile has only 128. Second, intermittency is concentrated, where zeros are absent from the median series but dominate the upper percentiles, as shown in Figure 5.

Table 3: **Statistics of demand datasets.** Counts and percentiles are over the whole converted corpus, real-world and synthetic together.

[1] Quantity	Value	[2] Series length	Points
Series	11,306,740	P25	128
Observations	48,355,437,247	P50	1,373
Converted sources	73	P75	8,761
Collected sources	93	P99	21,383
[3] Class	**Series**	**[4] Zero ratio**	**Share**
Smooth	6,340,416	P50	0.00
Erratic	4,141,168	P90	0.06
Intermittent	727,598	P99	0.98
Lumpy	97,558	P100	1.00

Figure 5: **Zeros in the corpus.** Zeros are absent from most series, and intermittency is therefore concentrated in a minority.



The intermittent and lumpy classes, the hard cases that motivate the corpus, are therefore a minority by count, and the generator of Section 3 is built to correct this imbalance by generating more series of the two rare classes.

2.3 Train-Test Split

Roles and their use. The five roles in Table 2 reduce to a single rule, namely that only evaluation suites are held out, while all other roles are training candidates until the leakage check below removes whatever overlaps the test side.

- **Evaluation** is the only role held out, as these three sources exist to be scored against, and using any part of them for training would invalidate the very measurement that they were published to provide.
- **Pretrain only** and **No split** are used for training in full, as neither withholds anything we are obliged to respect. What is removed from them is determined by the leakage check below rather than by the role.
- **Train only (held-out private)** is also used for training in full, since the hidden part was never published by its authors, and there is therefore no test set of our own that any part of it could contaminate.
- **Train and test** contributes *both* parts to training, as its test part is the competition's own rather than the benchmark we evaluate on, and any coincidence between the two is caught by the leakage check below.

This rule concerns how a source was published rather than what it contains, and it is therefore not sufficient on its own, since two sources can publish the same underlying series under different roles without any record of the overlap.

Train and test separation. Evaluation uses the held-out evaluation suites, and all remaining series are available for training. Separating them by dataset name is not sufficient, as the same underlying series is redistributed under different names across sources. We therefore separate the two sides at the level of the values themselves, where a redistributed copy cannot hide behind the name it was republished under. Any training candidate whose values match a test series is *removed rather than down-weighted*. Table 4 reports what the check leaves on each side, counted in series and in time points, together with what it removes from the corpus on the training side.

We apply the same measurement to the pretraining data of the models we compare against, as discussed in §5.1. That data was fixed before the benchmarks and was never filtered against them, and the comparison is therefore *not symmetric*, which we state rather than leave implicit. The synthetic corpus is untouched by this check, since it holds no real observation that the leakage check could match, and every one of its series is available to the training mixture in full.

Table 4: **Train and test separation.** Removed is what the leakage check takes out of training, and the shares are over the whole corpus.

	Series		Time points	
	Value	Share	Value	Share
Train	10,049,114	88.88%	47,419,572,862	98.06%
Test	899,618	7.96%	810,403,280	1.68%
Removed	358,008	3.17%	125,461,105	0.26%

Table 5: **Design of the synthetic generator by demand properties.** Each property has a component of its own in the sampling process, and each component is turned on or off for every series with its strength drawn at random, so that the properties combine freely rather than in the fixed groups that a fixed recipe would give.

Family	Property	Generating mechanism
Counts	Discreteness	Negative-binomial sampling from a multiplicative intensity
	Overdispersion	Dispersion parameter sampled per series
	Intermittency	Bernoulli mask over the intensity, targeted to a demand class
Calendar	Day-of-week profile	Multiplicative weekday factors
	Seasonality	Fourier components at annual and weekly periods
	Moving holidays	Lunar New Year, Chuseok, Easter, and Thanksgiving as shifting dates
Promotion	Uplift	Three-phase response of build-up, peak, and post-promotion dip
	Cannibalisation	Negative cross-effect on related series
	Price elasticity	Log-linear response of intensity to a sampled price path
Life-cycle	Product launch	Bass diffusion for adoption
	Discontinuation	Decaying tail with an absorbing zero state
Observation	Stock-out censoring	(s, S) inventory policy clipping observed sales below latent demand
	Reporting gaps	Contiguous missing spans

3 Synthetic Demand Dataset

The real-world corpus of Section 2 is broad but uneven, since several of the behaviours that decide whether a demand forecast is useful are thin in it, and one of them cannot appear in observed data at all. We therefore generate a synthetic corpus alongside the real one, designed to cover exactly those regions of demand behaviour.

3.1 Motivation

Why synthetic data. Open demand data under-represents several behaviours that matter operationally, and in some cases cannot represent them at all. Promotions and holidays are visible in the series only as unexplained level shifts. The calendar and the promotional plan that caused them are not published with the data. Product life-cycles are truncated, since a dataset released at one point in time rarely covers both the launch and the discontinuation of the same item. Most importantly, observed sales are a censored view of latent demand. When inventory runs out, a zero records an empty shelf rather than an absence of customers, and no real dataset distinguishes the two.

We therefore generate a synthetic corpus of 47,500 series and 59.7M points alongside the real one. Its purpose is not to substitute for real demand but to *cover the regions of demand behaviour that open data leaves thin*. The generating process is known, and it therefore records what observation alone cannot, as described in §3.2.

3.2 Generating Process

Generating process. A series is built in three stages, where a multiplicative intensity sets the expected level over time, a count process turns that intensity into integer demand, and an observation model decides what is actually recorded. Table 5 lists the properties the generator reproduces and the mechanism behind each. Every property is switched on independently with a sampled strength, which lets the corpus cover smooth seasonal demand and series dominated by long zero runs and abrupt life-cycle transitions, together with the mixtures that lie between the two extremes. The corpus is organised into eight generators, each configured to imitate a distinct operational setting rather than a distinct statistical family. This keeps the corpus interpretable, since a mixture can be described in terms of the business situations it emphasises. Figure 7 shows one series from each of the eight generators below.

The corpus records the pre-censoring latent demand, the underlying intensity, and a stock-out flag next to every observed value. This is a signal *no real dataset can provide*, since in observed sales data a zero caused by no customer and a zero caused by an empty shelf are indistinguishable in the file as the publisher releases it.

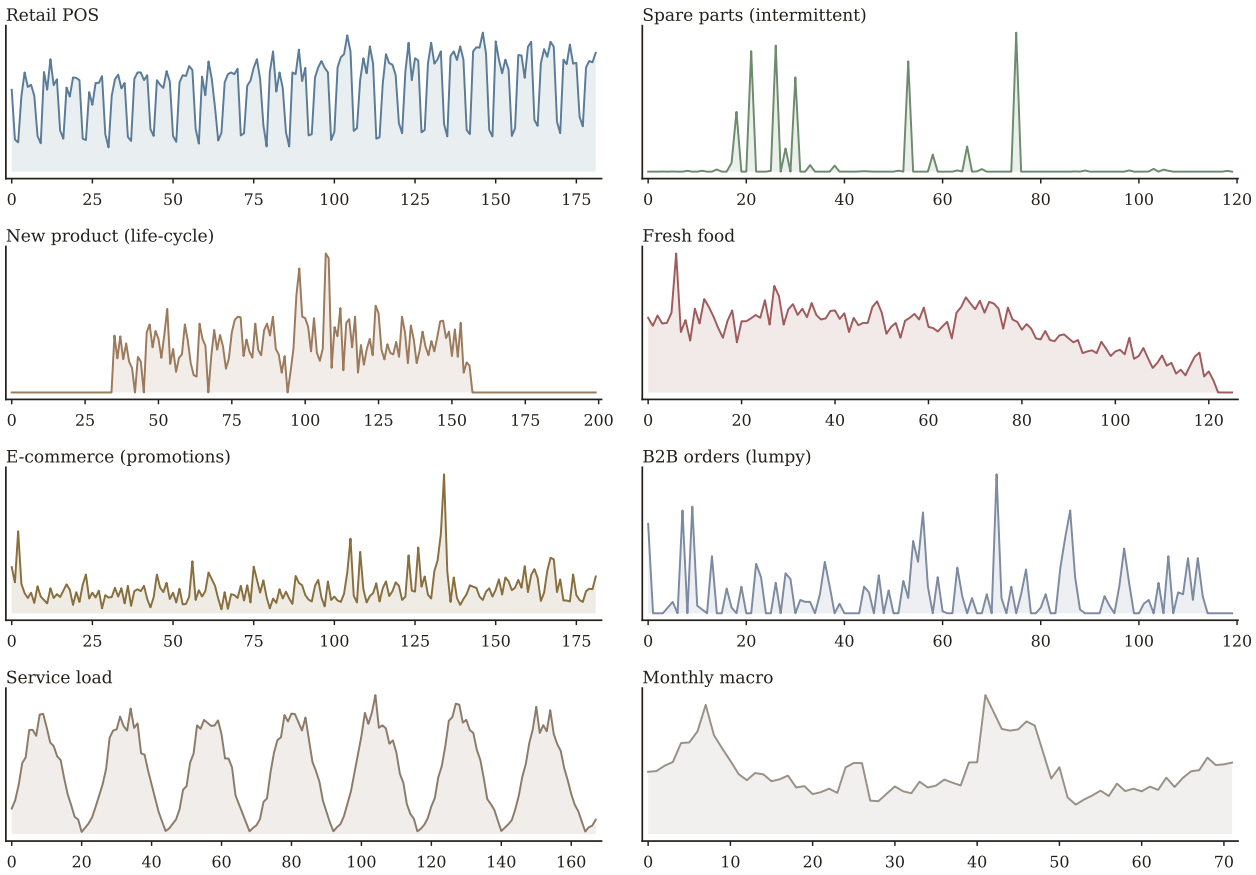


Figure 7: **Representative synthetic demand series.** One series per generator. Spare parts and B2B orders realise the intermittent and lumpy regimes that open data supplies only sparsely, new product realises a full launch-to-discontinuation life-cycle, and e-commerce carries explicit promotional responses of its own.

3.3 Validation and Use

Validation. Generation is controlled rather than left to an unconstrained sampler, as the Syntetos–Boylan class is an *input* to the generator, which produces series to hit a target class, and the realised classification is then checked against that target, where the two agree in 95.5% of cases. Seven further validation checks, covering class distribution, zero ratios, seasonality recovery, promotion response, censoring rate, length distribution, and determinism under a fixed seed, all pass on the final corpus, which confirms that the properties that the generator was written to produce are the properties the corpus actually carries rather than ones merely assumed from the generating process.

Use in training. The synthetic corpus contains no real observations by construction, where no value of it can match an evaluation series, and the leakage check therefore does not apply to it. It is therefore used in its entirety for training, with no held-out portion. All 47,500 series are available to every mixture, and the train and test separation reported in §2.3 concerns the real-world corpus alone, where a leaked series would otherwise be scored against itself.

4 EXAONE-Demand

Motivation. Demand series do not share one behaviour, since the corpus of Section 2 spans *smooth* seasonal sales, *intermittent* spare-part orders, *erratic* promotional spikes, and *lumpy* wholesale batches. The hard cases are a minority by count while being the reason the corpus exists, where one adapter fitted to all four spends most of its capacity on the easy majority, and we therefore let it specialise by demand behaviour rather than average across them.

Overview of EXAONE Demand. EXAONE Demand keeps a pretrained TSFM frozen and adapts it with a mixture-of-experts (MoE) adapter [61, 74], whose five experts are low-rank branches and whose mixing weights follow the demand behaviour of the input series. Four of them are routed, each tied to one of the four demand classes, and the

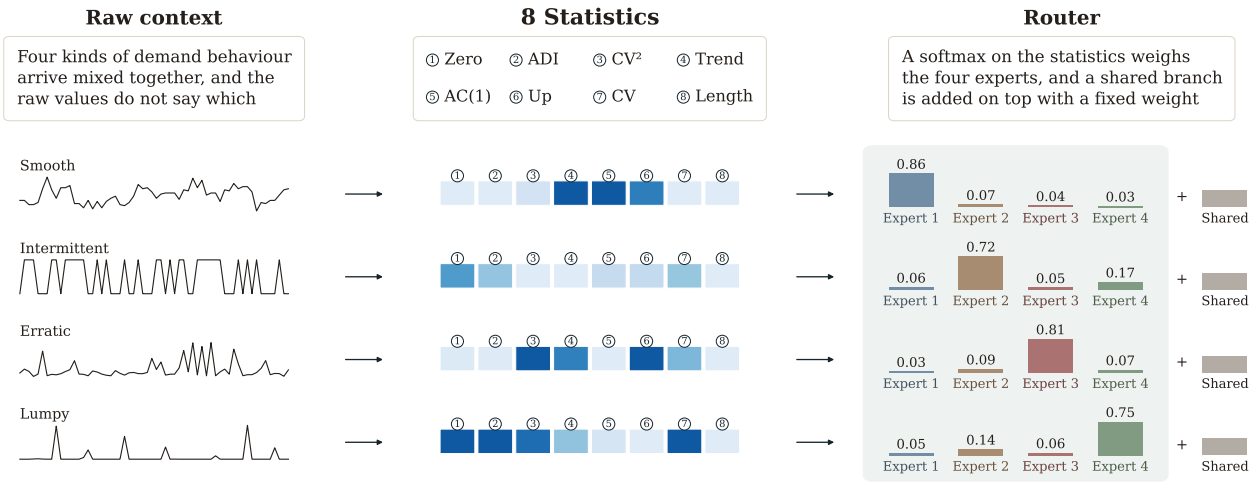


Figure 8: **EXAONE Demand on the four demand classes.** (1) *Raw context*: The window that the model receives, with no rescaling of its values. (2) *8 Statistics*: The scale-free summary that the window is reduced to. (3) *Router*: One weight per routed branch, with the always-on shared branch added at a fixed weight.

fifth is always on and shared by every series as a shared expert [15], so that it carries the correction that every demand series needs and leaves the routed four to carry only the differences between the classes.

The pipeline has five stages, and we describe each in turn:

- **Demand statistics** (§4.1): We summarise the raw context window into eight scale-free statistics.
- **Continuous demand membership** (§4.2): We turn ADI and CV^2 into a membership over the four demand classes.
- **Routing** (§4.3): We route on the eight statistics and supervise the router with that membership.
- **Mixture of low-rank branches** (§4.4): We mix the shared and the four routed branches in every frozen projection.
- **Training** (§4.5): We train only the low-rank branches and the router, keeping the backbone frozen.

Figure 8 draws the whole pipeline, with four real series, one from each of the four demand classes, and the statistics and the membership that each of them implies at the input of the adapter.

4.1 Stage 1: Demand Statistics

The router reads eight statistics of the raw series, and each of them answers one question about the demand behaviour that separates the four classes. We list them in the order they enter the router input:

[1] **Zero share** (Zero): The fraction of the window with no demand, which is the plainest sign of intermittency.
[2] **Average inter-demand interval** (ADI): The mean spacing between two non-zero periods, which is the first axis of the Syntetos–Boylan classification, separating smooth from intermittent demand.
[3] **Squared coefficient of variation** (CV^2): The dispersion of the non-zero values around their own mean, which is the second axis of the same scheme, separating steady from erratic order sizes.
[4] **Trend correlation** (Trend): The correlation between the series and time, which distinguishes a product in its launch or in its decline phase from one that has held a stable level.
[5] **First-order autocorrelation** (AC(1)): How much one period predicts the next, which tells a seasonal or otherwise persistent series apart from one whose movements are close to noise.
[6] **Fraction of upward steps** (Up): The share of consecutive pairs that increase, which describes the shape of the path the demand takes rather than its level or its spread.
[7] **Coefficient of variation** (CV): The dispersion of the whole window, zeros included, which reacts to the spikes that the dispersion of the non-zero values alone would miss.
[8] **Length** (Length): The number of observed periods, which says how much evidence the other seven rest on.

The first three are ratios over counts, the next three describe the shape of the path, and the last two describe its spread and its size, and the eight together therefore cover both what the two classical statistics of §4.2 rest on and what they leave out. We now define each of them, from the raw window to the vector the router receives.

Table 6: **Example router input for four demand series.** Each row is the vector s of Equation 4 as the router receives it, and the four series are drawn from four different real-world sources and are of four different lengths.

Class	Source	8 Statistics							
		Zero	**ADI**	**CV^2**	**Trend**	**AC(1)**	**Up**	**CV**	**Length**
Smooth	Citi Bike	0.00	0.23	0.04	−0.16	0.36	−0.87	0.10	0.56
Intermittent	Port calls	0.98	1.26	0.00	−0.09	−0.02	−0.95	0.67	0.70
Erratic	SKU orders	0.03	0.24	0.32	−0.04	0.10	−0.66	0.28	0.51
Lumpy	Retail sales	0.86	0.69	0.14	−0.01	0.00	−0.79	0.47	0.63

Let $x \in \mathbb{R}^L$ be the raw context window of one series and let $m \in \{0,1\}^L$ mark its observed entries, with $n = \sum_t m_t$. We first decide which of those entries count as a demand event, and because units differ by three orders of magnitude across the corpus, the test is relative to the scale of the series rather than absolute,

$$\epsilon = 10^{-3} \cdot \frac{1}{n} \sum_t m_t |x_t|, \qquad z_t = m_t \cdot \mathbb{1}[\,|x_t| > \epsilon\,], \qquad n_z = \sum_t z_t. \tag{1}$$

From these we form the two axes of the Syntetos–Boylan classification [66], namely the average inter-demand interval and the squared coefficient of variation of the non-zero values,

$$\mathrm{ADI} = \frac{n}{n_z}, \qquad \mu_z = \frac{1}{n_z} \sum_t z_t |x_t|, \qquad \mathrm{CV}^2 = \frac{1}{n_z \mu_z^2} \sum_t z_t \left(|x_t| - \mu_z\right)^2. \tag{2}$$

Four further quantities describe the shape of the series. With $\bar{x}$ the mean over observed entries, $\tilde{x}_t = m_t(x_t - \bar{x})$ the centred series, and $\tilde{t}$ the centred time index, these are the trend correlation, the first-order autocorrelation, the fraction of upward steps, and the overall coefficient of variation,

$$\rho_{\mathrm{trend}} = \frac{\langle \tilde{t}, \tilde{x} \rangle}{\|\tilde{t}\|\,\|\tilde{x}\|}, \quad \rho_1 = \frac{\langle \tilde{x}_{1:L-1}, \tilde{x}_{2:L} \rangle}{\|\tilde{x}_{1:L-1}\|\,\|\tilde{x}_{2:L}\|}, \quad u = \frac{1}{n-1} \sum_t \mathbb{1}[x_{t+1} > x_t], \quad \mathrm{CV} = \frac{\sigma}{|\bar{x}|}. \tag{3}$$

The router input is the concatenation of eight numbers, where the three ratio-valued ones, namely entries 2, 3, and 7, and the length in entry 8 are compressed on a logarithmic axis so that no single statistic dominates the first layer,

$$s = \left[\, 1 - \tfrac{n_z}{n},\ \tfrac{1}{3}\log(1{+}\mathrm{ADI}),\ \tfrac{1}{3}\log(1{+}\mathrm{CV}^2),\ \rho_{\mathrm{trend}},\ \rho_1,\ 2u-1,\ \tfrac{1}{3}\log(1{+}\mathrm{CV}),\ \tfrac{1}{8}\log n \,\right] \in \mathbb{R}^8. \tag{4}$$

Table 6 shows s for four series taken from the corpus, one from each of the four demand classes. The classes separate on different entries, which is why eight numbers are kept rather than the two that define them. The smooth and erratic series agree on the zero share and on the interval, and are separated by the dispersion of their values, while the intermittent and lumpy series are told apart by the interval and by that same dispersion together.

Every entry of s is invariant to the unit of the series. A router that can see the magnitude of the values learns to recognise the *source* rather than the *behaviour*, and a source it has not seen then falls outside everything it has learned. The statistics are computed once per series, before the encoder, and the same eight numbers are handed to every adapted layer, so that no layer re-derives them from the hidden state that the backbone passes to it.

4.2 Stage 2: Continuous Demand Membership

The classical scheme of the intermittent demand literature [14, 65, 66] assigns a series to one of four classes by cutting Equation 2 at ADI = 1.32 and CV^2 = 0.49, as shown in Table 7. However, demand series do not respect those cuts. They spread *continuously* across the two axes, and a series just past a boundary is not a different kind of object from the one that lies just before it. Series at ADI = 1.30 and 1.34 are nearly alike, yet the cuts split them.

Table 7: **Classical demand classes.** A low ADI means that demand arrives often, and a high CV^2 means that its size varies widely.

	CV^2 < 0.49	CV^2 ≥ 0.49
ADI < 1.32	**Smooth** Frequency ↑, Variability ↓	**Erratic** Frequency ↑, Variability ↑
ADI ≥ 1.32	**Intermittent** Frequency ↓, Variability ↓	**Lumpy** Frequency ↓, Variability ↑

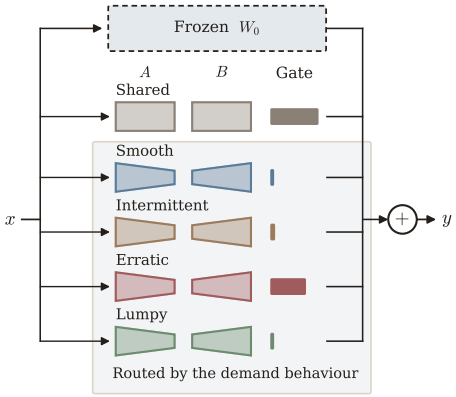


Figure 9: **One adapted projection.** The four routed branches are weighed by the gate, and the shared branch is always on.

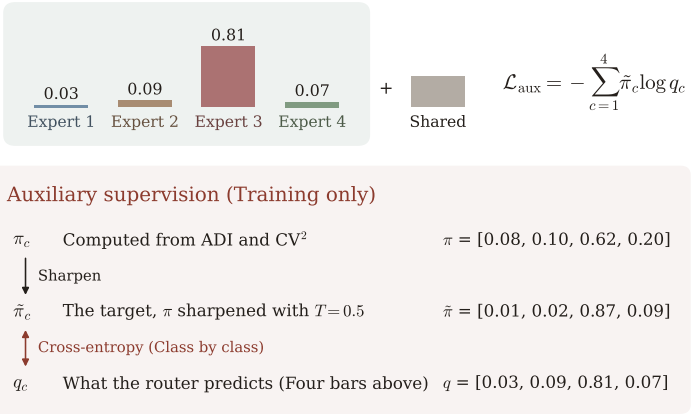


Figure 10: **Auxiliary supervision on one erratic series.** The bars are the router output, and the chain below turns the membership into the target that the cross-entropy pulls that output towards during training.

We therefore keep the two axes but soften the cuts, and since both quantities are ratios, we measure the distance to each cut on a logarithmic axis, where the gap between ADI = 1.2 and 1.4 carries the same weight as the gap between 8.0 and 9.3,

$$a = \sigma\left(\frac{\log \text{ADI} - \log 1.32}{\tau_a}\right), \qquad c = \sigma\left(\frac{\log \text{CV}^2 - \log 0.49}{\tau_c}\right), \tag{5}$$

with σ the logistic function and τ_a, τ_c the widths of the cuts. Here a reads as how *intermittent* the series is and c as how *erratic*, and treating the two axes as independent gives a membership whose entries sum to one,

$$\pi = \left[\,(1-a)(1-c),\ a(1-c),\ (1-a)c,\ ac\,\right] = [\pi_{\text{smooth}}, \pi_{\text{inter}}, \pi_{\text{erratic}}, \pi_{\text{lumpy}}]. \tag{6}$$

A series that is clearly one thing gets a membership close to a corner, while a series between two behaviours keeps *both* of them. The lumpy example of Table 6 has $\pi = (0.00, 0.46, 0.00, 0.53)$, which says it is lumpy by a narrow margin over intermittent, a split between the two classes that a hard label would have thrown away.

4.3 Stage 3: Routing

Each of the four routed branches is meant to carry one demand behaviour, and a rule has to decide how much of each branch a given series receives, which makes the membership of Equation 6 the obvious candidate for that rule. It rests on two of the eight statistics, however, and has to be recomputed from ADI and CV^2 for every series that arrives. Therefore, we learn that rule instead, as a small network that reads the whole vector s and that we call the *router*, while the membership is kept as a supervision signal rather than as the mixture itself. The auxiliary term of §4.5 makes that arrangement precise, and it is that term which ties each routed branch to one demand class.

That router is a two-layer network applied to s, followed by a softmax,

$$q = \text{softmax}\left(\frac{W_2\,\phi(W_1 s + b_1)}{\tau}\right) \in \Delta^3, \qquad W_1 \in \mathbb{R}^{h\times 8},\ W_2 \in \mathbb{R}^{4\times h}, \tag{7}$$

where ϕ is a GELU nonlinearity, τ is a temperature, and h is the hidden width. The router produces four weights, one per demand class of Equation 6. The shared branch is not routed at all, as it takes a fixed weight w while the routed branches divide whatever remains,

$$g = \left[\,w,\ (1-w)\,q_1,\ (1-w)\,q_2,\ (1-w)\,q_3,\ (1-w)\,q_4\,\right] \in \Delta^4. \tag{8}$$

One gate is produced per series rather than per token, and every token of a series is adapted by the same mixture.

Figure 9 draws what one adapted projection then holds, namely the frozen weight, the shared branch, and the four routed ones that the gate weighs. Each adapted layer carries its own router, and every one of them reads the same vector s and decides its own mixture from it. The router is small on purpose and cannot learn anything beyond a partition of the eight-dimensional space of statistics, which is all the design asks of it and all the supervision teaches.

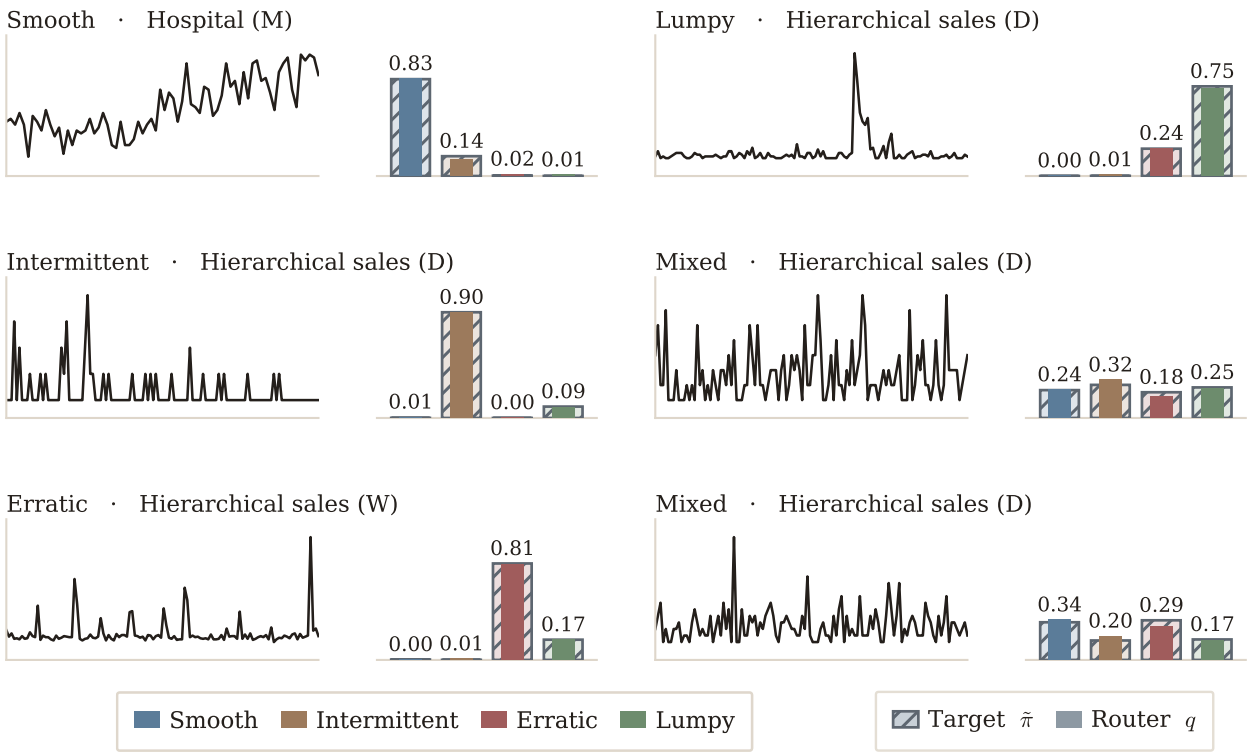


Figure 11: **What the router produces on real demand series.** Each panel holds a context window and the weight that the gate gives to each of the four routed branches, against the target that supervision asks for, where the four bars carry the smooth, intermittent, erratic, and lumpy branches of the mixture in that same order.

Nothing in Equation 7 ties routed branch c to any particular behaviour. Left alone the routed branches are *interchangeable*, and whichever partition the gate settles on is an *accident of initialisation*. We therefore supervise the gate with the membership of Equation 6, using a cross-entropy between the two distributions,

$$\mathcal{L} = \mathcal{L}_{\text{forecast}} + \lambda \cdot \mathcal{L}_{\text{aux}}, \qquad \mathcal{L}_{\text{aux}} = -\sum_{c=1}^{4} \tilde{\pi}_c \log q_c. \tag{9}$$

The loss is written on q rather than on g, which leaves the shared branch out of it and supervises only the four routed branches. That is what gives routed branch c a meaning of its own, in that branch 1 is pulled toward smooth series and branch 2 toward intermittent ones, while a series that is 0.6 lumpy and 0.4 intermittent asks for both in that proportion.

The target $\tilde{\pi}$ is a sharpened form of the membership of Equation 6,

$$\tilde{\pi}_c = \frac{\pi_c^{1/T}}{\sum_{c'} \pi_{c'}^{1/T}}, \tag{10}$$

where $T < 1$ is the sharpening temperature. We sharpen because series near the cuts of Equation 2 carry a membership that is close to a tie, and a target such as $[0, 0.40, 0, 0.60]$ is too weak to settle which branch should lead. The alternative is a hard label and a negative log-likelihood on the assigned class, which pulls q toward a one-hot vector. We do not use it, because it contradicts the premise of Equation 6, namely that a demand series is a *mixture of behaviours* rather than a *member of one class*.

Figure 10 follows one erratic series through that chain, from the membership we measure to the target we sharpen and the weights the router finally produces. The auxiliary term is a *training signal only*, and at inference the router reads the eight statistics of the raw window and no membership is computed, as only the loss needs it.

Figure 11 puts the trained router of EXAONE Demand on six real series to show what those weights look like once training is done. The four panels named by a class are the series on which the router is the most decisive, and on them the weight it produces sits almost exactly on the target that the supervision asks for. The two panels named *Mixed* are the series on which it is the least decisive, and the target there is spread across three branches while the router spreads its weight in the same way, which is what a series that mixes two behaviours is meant to look like at the gate.

4.4 Stage 4: Mixture of Low-Rank Branches

We replace six projections in each encoder block, namely the four attention projections and the two feed-forward projections, and leave the rest of the block untouched. Each adapted layer keeps its frozen weight W_0 and adds $E = 5$ low-rank branches mixed by g,

$$y = W_0 x + \sum_{e=0}^{4} g_e \cdot \frac{\alpha}{r_e} \cdot B_e A_e x, \qquad A_e \in \mathbb{R}^{r_e \times d_{\text{in}}},\ B_e \in \mathbb{R}^{d_{\text{out}} \times r_e}. \tag{11}$$

Branch 0 is the shared one and branches 1 to 4 are the routed ones. The shared branch is given the larger rank, so that it alone holds as much capacity as a plain low-rank adapter, while each routed branch adds a small correction on top of it. A shared branch that is too small would leave part of the common correction to the routed branches, and that residue would blur the routing that the supervision of Stage 3 asks for. The scale α / r_e follows the low-rank convention and is applied per branch, so that a change of rank does not change the scale at which a branch enters the sum.

Written naively, Equation 11 forms one output tensor per branch. We instead stack the branches along the rank axis and evaluate the sum with two matrix products,

$$y = W_0 x + \bar{B}\left((\bar{A} x) \odot (g \otimes \mathbb{1}_r)\right), \qquad \bar{A} = \begin{bmatrix} A_0 \\ \vdots \\ A_4 \end{bmatrix}, \quad \bar{B} = \begin{bmatrix} B_0 & \cdots & B_4 \end{bmatrix}, \tag{12}$$

where the only intermediate tensor has width $\sum_e r_e$ rather than $E\, d_{\text{out}}$.

4.5 Stage 5: Training

Only A_e, B_e, and the router of Equation 7 receive gradients. Everything else stays frozen, including the embedding and the normalisation layers. The loss is Equation 9, and the weight λ is held fixed for every run that we report.

The three choices below are what keep the four routed branches apart from each other, and each of them does so in a different way, namely how the router starts, what it reads, and how much weight the shared branch is given.

Router starts at random. With $W_2 = 0$ the softmax of Equation 7 is exactly uniform for every input, every routed branch receives the same gradient, and the symmetry between them never breaks. A fixed uniform mixture of E branches of rank r equals a single adapter of rank Er by Equation 11, and the architecture then reduces to the thing it was meant to improve on. Drawing W_2 at random costs nothing, since the adapter output is multiplied by $B_e = 0$ at step zero and the adapted layer therefore returns $W_0 x$ exactly, no matter which weights the router assigns.

Router reads the raw series rather than the hidden state. A gate applied to hidden states learns to recognise something the backbone encodes for itself, and its softmax flattens for the same reason as above. The raw series gives the router information that the encoder *never sees*, since the encoder is fed a normalised window.

Shared weight balances the two parts. The weight w of Equation 8 decides how much of the adapter is common and how much is routed. Near zero the routed branches carry almost everything and near one they carry almost nothing, and we therefore set w between the two, where both parts contribute, at $w = 0.5$ in every run that we report.

5 Experiments

5.1 Datasets and Metrics

Evaluation suite. We evaluate on 22 held-out datasets, none of which takes any part in training, as verified by the leakage check of Section 2 at the level of *raw values* rather than *dataset names*. Every model is scored on the same window, the same horizon, and the same quantile levels, with no per-model tuning of any kind.

Metrics. We report the six metrics of Table 8, namely the mean absolute scaled error (MASE), the normalised deviation (ND), the weighted quantile loss (WQL), the mean absolute percentage error (MAPE), the mean absolute error (MAE), and the mean scaled interval score (MSIS), and we aggregate every one of them across the 22 datasets by geometric mean, which gives every dataset the same share of the aggregate whatever level its errors sit at.

Let y_t be the observed value at step t of a horizon of length h, let $\hat{y}_t$ be the median forecast, let $\hat{y}_t^{(q)}$ be the forecast at quantile level q, and let $x_1, \ldots, x_n$ be the training portion of the same series at seasonal period m, which the first and the last of the six use alone. The six of them are written as follows,

$$\text{MASE} = \frac{\frac{1}{h}\sum_{t=1}^{h} |y_t - \hat{y}_t|}{\frac{1}{n-m}\sum_{t=m+1}^{n} |x_t - x_{t-m}|}, \qquad \text{ND} = \frac{\sum_{t=1}^{h} |y_t - \hat{y}_t|}{\sum_{t=1}^{h} |y_t|}, \tag{13}$$

$$\mathrm{WQL} = \frac{\sum_{q \in Q} \sum_{t=1}^{h} 2\left(q\,[\,y_t - \hat{y}_t^{(q)}\,]_+ + (1-q)\,[\,\hat{y}_t^{(q)} - y_t\,]_+\right)}{|Q| \sum_{t=1}^{h} |y_t|}, \qquad \mathrm{MAPE} = \frac{1}{h} \sum_{t=1}^{h} \frac{|y_t - \hat{y}_t|}{|y_t|}, \tag{14}$$

$$\mathrm{MAE} = \frac{1}{h} \sum_{t=1}^{h} |y_t - \hat{y}_t|, \qquad \mathrm{MSIS} = \frac{\frac{1}{h} \sum_{t=1}^{h} \left(u_t - l_t + \frac{2}{\alpha}\left[\,l_t - y_t\,\right]_+ + \frac{2}{\alpha}\left[\,y_t - u_t\,\right]_+\right)}{\frac{1}{n-m} \sum_{t=m+1}^{n} |x_t - x_{t-m}|}, \tag{15}$$

where Q is the set of nine quantile levels that both of our models emit, $[\cdot]_+$ keeps the positive part of its argument, and l_t and u_t are the bounds of the central $1-\alpha$ interval at $\alpha = 0.2$. Each of the six divides the error by a different quantity, and that divisor is what the metric is about. All six are non-negative and unbounded above, and lower is better on each:

- ***MASE***: How much better than doing nothing a model is, since the divisor is the error that a seasonal naive forecaster makes on the history of the same series. A value above one says that repeating last season would have been the better choice of the two, and every row of Table 8 sits above one on the present evaluation suite.
- ***ND***: The share of the demand that the forecast misplaces, since the divisor is the total volume of the horizon. A value of 0.14 reads as fourteen units misplaced for every hundred demanded, the currency a planner stocks in.
- ***WQL***: How good the whole predictive distribution is rather than its middle alone, since the error at each quantile level is charged asymmetrically. Missing above the 0.9 quantile costs nine times as much as missing below it.
- ***MAPE***: The error as a fraction of the demand at each step taken separately, which is the most familiar of the six and the least suited to this setting. A value of 0.26 reads as a quarter of the demand missed on the average period.
- ***MAE***: The average absolute error, and the only one of the six that keeps the unit of the series. Its level says nothing across datasets, while its ordering across the models of one suite still says what the others say.
- ***MSIS***: How good the interval is rather than the point or the distribution, charging the width of the central 80% band plus a penalty for every observation outside it. It is the one metric that widening the band alone cannot win.

Average rank and win rate. To separate *how much* a model is ahead from *how often* it is ahead, we report two further quantities beside the six metrics, both of which read the per-dataset MASE rather than the aggregate of it. An aggregate is a single number that a handful of datasets can carry, and a model that is far ahead on three datasets and behind on the other nineteen can lead on it, which is the reading that these two are there to rule out.

The *average rank* of a model is its mean position across the 22 datasets, where a model that is second on every one of them scores 2 however wide or narrow the gaps around it are. The *win rate* is the share of the pairwise comparisons that a model wins, and with 38 other rows and 22 datasets there are 836 comparisons for each of them. A model wins one of those when its MASE on that dataset is the lower of the two, and a tie counts as a win for neither model.

5.2 Experimental Setup

Baselines. We compare against the 36 TSFMs of Table 8 and the frozen backbone that we adapt, all *run by us* rather than *quoted from their papers*. The comparison covers Chronos-Bolt (Base, Small) and Chronos-T5 (Base, Small) [4], Chronos-2 and Chronos-2 (Synthetic) [3], TiRex-1.1 [5], TimesFM-1.0, TimesFM-2.0 and TimesFM-2.5 [17], Moirai-1.1-R (Small, Base, Large) and Moirai-2.0-R [72], Timer-S1 [47], Sundial [46], Toto (Open Base) [13], TabPFN-TS [31], FlowState and Granite FlowState-R1 [27], TTM-R1 and TTM-R2 [21], PatchTST-FM-R1 and Granite PatchTST-FM-R1 [55], Kairos (10M, 23M, 50M) [22], TempoPFN [52], Reverso [23], VisionTS [10], Lag-Llama [58], CleanTS [20], and YingLong (6M, 50M, 110M, 300M) [70]. No baseline is selected by how well it does on the 22 datasets, and every model, ours included, is scored under the same protocol, with no chance to tune on it at any stage of the study that this section reports and the appendix records in full.

Implementation. The adapter of Section 4 is attached to the four attention projections and the two feed-forward projections of every encoder block, and the backbone stays frozen throughout. The shared branch is given the larger rank and half of the mixture weight, and the router is trained with the auxiliary term of Equation 9 alongside the forecast loss. Appendix A records every hyperparameter behind the tables and figures of this section.

5.3 Two Versions of EXAONE Demand

We build two models that differ only in their training data. **EXAONE Demand**, the model of Section 4, is trained on real-world and synthetic demand together, and **EXAONE Demand (Synthetic)** on the synthetic corpus alone, with no real-world series taking part at any point of training. The second model exists because open demand data carries licences that a model trained on it inherits, while one trained on series we generated *ourselves* inherits none of them. The question is whether the synthetic corpus can *stand on its own* rather than only *fill gaps*.

Table 8: **Comparison with TSFMs.** Both of our models are shown against every baseline we ran and against the frozen backbone that we adapt, on the 22 evaluation datasets, with the rows ordered and numbered (#) by MASE.

#	Model	Ranking		Error metrics					
		Avg. rank	Win rate	MASE	ND	WQL	MAPE	MAE	MSIS
1	EXAONE Demand	**4.09**	**91.9%**	**1.0667**	**0.1409**	**0.1138**	**0.2565**	**60.24**	10.70
2	EXAONE Demand (Synthetic)	5.55	88.0%	1.0742	0.1420	0.1147	0.2583	60.73	**10.63**
3	TiRex-1.1 [5]	7.64	82.5%	1.0818	0.1451	0.1161	0.2911	62.06	10.75
4	Chronos-2 [3]	8.59	80.0%	1.0885	0.1497	0.1406	0.3010	64.02	14.48
5	EXAONE Backbone (Zero-shot)	7.95	81.7%	1.1050	0.1445	0.1169	0.2609	61.82	11.41
6	TimesFM-2.5 [17]	10.05	76.2%	1.1073	0.1486	0.1215	0.3032	63.58	12.31
7	Chronos-Bolt (Base) [4]	11.05	73.6%	1.1122	0.1493	0.1191	0.2941	63.84	11.14
8	Timer-S1 [47]	12.27	70.3%	1.1197	0.1468	0.1171	0.2997	62.77	12.08
9	Reverso [23]	11.27	73.0%	1.1291	0.1510	0.1510	0.2773	64.60	45.17
10	Chronos-Bolt (Small) [4]	14.00	65.8%	1.1331	0.1485	0.1193	0.2998	63.53	11.57
11	Chronos-T5 (Base) [4]	8.68	79.8%	1.1336	0.1494	0.1219	0.2712	63.90	13.33
12	Toto (Open Base) [13]	11.73	71.8%	1.1399	0.1495	0.1214	0.2677	63.93	11.64
13	Chronos-2 (Synthetic) [3]	15.95	60.6%	1.1515	0.1594	0.1452	0.3088	68.17	16.11
14	Moirai-2.0-R [72]	12.05	70.9%	1.1586	0.1531	0.1260	0.2802	65.48	13.56
15	Chronos-T5 (Small) [4]	12.27	70.3%	1.1623	0.1524	0.1249	0.2751	65.18	13.87
16	Sundial [46]	15.68	61.4%	1.1690	0.1543	0.1325	0.3006	66.00	19.26
17	TabPFN-TS [31]	20.23	49.4%	1.2172	0.1719	0.1353	0.3520	73.52	12.21
18	FlowState [27]	19.23	52.0%	1.2363	0.1705	0.1357	0.3422	72.94	12.14
19	Granite FlowState-R1 [27]	18.68	53.5%	1.2419	0.1710	0.1363	0.3469	73.12	12.67
20	Moirai-1.1-R (Large) [72]	18.23	54.7%	1.2572	0.1640	0.1322	0.3027	70.16	12.51
21	Moirai-1.1-R (Base) [72]	19.09	52.4%	1.2724	0.1638	0.1331	0.3108	70.06	12.67
22	TimesFM-1.0 [17]	20.05	49.9%	1.2794	0.1727	0.1427	0.3448	73.87	16.27
23	PatchTST-FM-R1 [55]	21.23	46.8%	1.2984	0.1693	0.1384	0.3312	72.41	12.91
24	Moirai-1.1-R (Small) [72]	22.50	43.4%	1.3570	0.1753	0.1423	0.3230	74.98	13.58
25	TempoPFN [52]	21.68	45.6%	1.3968	0.1794	0.1475	0.3716	76.73	19.61
26	Granite PatchTST-FM-R1 [55]	23.41	41.0%	1.4061	0.1790	0.1493	0.3481	76.54	16.25
27	Kairos (50M) [22]	22.23	44.1%	1.4641	0.1905	0.1572	0.3648	81.46	18.97
28	Kairos (23M) [22]	26.00	34.2%	1.5391	0.2081	0.1737	0.3730	89.02	20.87
29	TTM-R1 [21]	27.91	29.2%	1.5714	0.2014	0.2014	0.4047	86.13	62.86
30	TTM-R2 [21]	29.73	24.4%	1.6399	0.2125	0.2125	0.4387	90.89	65.60
31	Kairos (10M) [22]	26.91	31.8%	1.6463	0.2067	0.1705	0.3909	88.42	21.05
32	VisionTS [10]	30.27	23.0%	1.9069	0.2444	0.2444	0.5455	104.55	76.28
33	Lag-Llama [58]	29.73	24.4%	2.0328	0.2437	0.2078	0.4339	104.24	24.97
34	TimesFM-2.0 [17]	33.14	15.4%	2.4684	0.2548	0.1989	0.4632	108.98	20.09
35	CleanTS [20]	33.86	13.5%	3.1566	0.3705	0.3112	0.7209	158.48	45.16
36	YingLong (6M) [70]	36.00	7.9%	3.7329	0.4672	0.4051	0.8661	199.83	73.96
37	YingLong (110M) [70]	36.82	5.7%	3.8443	0.4773	0.4289	0.8865	204.14	88.79
38	YingLong (300M) [70]	36.86	5.6%	3.8491	0.4879	0.4319	0.8870	208.69	85.43
39	YingLong (50M) [70]	37.41	4.2%	3.8853	0.4861	0.4348	0.8876	207.93	87.12

5.4 Main Results

Table 8 reports both of our models against every baseline we ran, on the 22 evaluation datasets and under the protocol of §5.1. Both of them lead the table on every one of the six metrics as well as on the average rank and the win rate, and neither was tuned against the suite that scores them. The two take the first and the second place on five of the six metrics, and they divide those two places between themselves on MSIS, where the synthetic-only model is the better of the pair by a margin of only 0.07, and we therefore read MSIS as a tie between our two models rather than as a win.

EXAONE Demand wins 91.9% of its 836 pairwise comparisons against the other 38 rows, where the strongest baseline wins 82.5% of the comparisons among its own peers. The synthetic-only model wins 88.0% and stays ahead of every baseline in the table, which shows that the synthetic corpus can carry a model on its own.

Table 9: **Per-dataset MASE on the 22 evaluation datasets.** Lower is better, the letter in brackets is the sampling interval, and the five baselines are the strongest of the 36 by their aggregate over the suite. Real+Synth is the model trained on real-world and synthetic demand, while Synth is the one trained on the synthetic corpus alone.

Dataset	EXAONE Demand		TSFM Baseline				
	Real+Synth	**Synth**	**TiRex-1.1** [5]	**Chronos-2** [3]	**TimesFM-2.5** [17]	**Chronos-Bolt (Base)** [4]	**Timer-S1** [47]
Bitbrains (fast storage) (H)	1.020	1.008	1.016	**0.995**	1.120	1.082	1.126
Bitbrains (random) (H)	**5.829**	5.832	5.859	5.853	5.951	5.926	5.955
BizITObs L2C (H)	0.441	0.445	0.487	0.439	0.520	0.428	**0.407**
Car parts (M)	**0.828**	0.847	0.916	0.900	0.942	0.903	0.893
Electricity (D)	**1.375**	1.383	1.466	1.415	1.436	1.513	1.456
Electricity (H)	1.010	1.009	0.912	0.888	0.942	**0.886**	0.897
Electricity (W)	1.494	1.468	1.478	**1.456**	1.491	1.517	1.614
Hierarchical sales (D)	**0.746**	0.761	0.768	0.768	0.786	0.765	0.776
Hierarchical sales (W)	**0.719**	0.722	0.747	0.745	0.748	0.764	0.755
Hospital (M)	**0.759**	0.765	0.785	0.807	0.763	0.828	0.839
Loop Seattle (D)	0.889	0.891	0.900	0.928	**0.881**	0.909	0.896
Loop Seattle (H)	0.817	0.819	0.820	0.816	0.880	0.898	**0.808**
M4 daily (D)	3.079	**3.046**	3.063	3.164	3.202	3.070	3.336
M4 hourly (H)	0.763	0.805	**0.708**	0.800	0.724	0.874	0.797
M4 monthly (M)	**0.909**	0.925	0.938	0.943	0.963	0.978	1.038
M4 quarterly (Q)	1.149	1.150	**1.118**	1.177	1.185	1.210	1.259
M4 weekly (W)	2.066	2.150	**1.973**	2.070	1.980	2.154	2.442
M4 yearly (Y)	**3.165**	3.217	3.238	3.256	3.635	3.328	3.692
M-dense (D)	0.645	0.640	0.678	0.681	0.652	0.676	**0.631**
M-dense (H)	0.786	0.783	0.802	0.807	0.830	0.799	**0.776**
Restaurant (D)	**0.676**	0.677	0.704	0.713	0.696	0.730	0.717
SZ taxi (H)	**0.566**	0.567	0.582	0.591	0.573	0.580	0.598
1st Count	10	1	3	2	1	1	4
2nd Count	2	13	1	2	3	1	0

5.5 Results by Dataset

An aggregate can hide a group of datasets on which a model loses while it still wins on the geometric mean, and a demand model is exactly the case where that matters, since the hard classes are a minority of the suite. Table 9 therefore reports every one of the datasets separately rather than folded into a single number.

The spread across the rows of that table is wider than the spread across its columns, since some datasets are several times harder than the rest for every baseline we ran, and a few rows of that kind move the aggregate further than any architecture choice of Section 4 moves it, as Bitbrains (random), M4 yearly, and M4 daily show.

Neither of our models is last on any of the 22 datasets, each of them is ahead of all 36 baselines on 7 of them, and the real-world model is ahead of the synthetic-only one on 16 of the 22. The gap between the two is therefore a lead held across the suite rather than the work of a handful of rows on which one of them happens to do unusually well, and the same holds of the gap between either of them and the strongest of the 36 baselines in Table 8.

5.6 Forecast Visualization

Figure 12 places EXAONE Demand beside the two strongest baselines of Table 8, TiRex-1.1 and Chronos-2, on six series of the evaluation suite, and our model reaches the lowest MASE of the three on every one of them. On the spare-part and hospital series, the two baselines forecast a return of demand that never arrives, while our model stays at zero on the first and holds the level on the second. On the two electricity series, demand collapses to zero within the horizon and TiRex-1.1 keeps repeating the cycle it has read, while our model follows the collapse. On the two hourly series, M-dense and M4 hourly, our model tracks the height of the sharp peaks most closely. The panels are chosen to show what the adaptation does when it works, and a selected curve cannot carry the claim that the aggregate carries.

5.7 Effect of Training Data

To isolate what real-world demand contributes, we compare runs that share the architecture, learning rate, target set, and seed, and differ only in whether *real-world series take part in training*. The left panel of Figure 13 sweeps the share of synthetic series in the mixture from 0.05 to 1.0, where the last setting is the synthetic corpus alone. The curve has an interior minimum at the share we adopt, and it rises sharply once no real-world series is left in the mixture, yet both of our models stay well below TiRex-1.1, the strongest baseline of Table 8. The right panel places our two

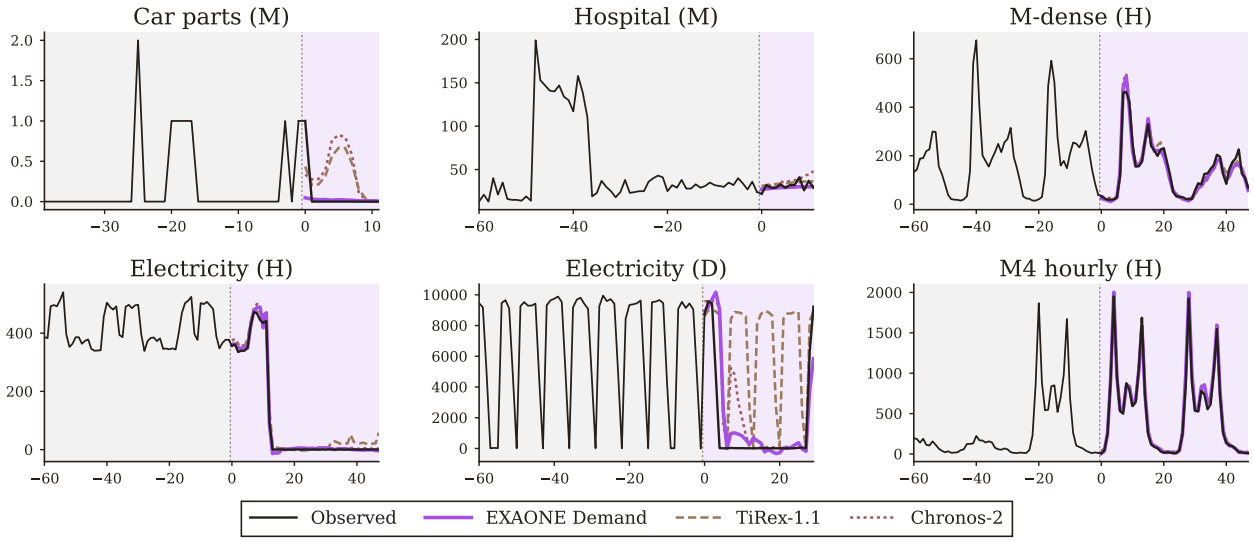


Figure 12: **Forecasts on six demand series.** Each line is the median forecast of one model, and EXAONE Demand reaches the lowest MASE of the three on all six series. On Car parts (M) and Electricity (D), where demand falls to zero, TiRex-1.1 and Chronos-2 keep forecasting a positive level, while EXAONE Demand follows the zeros.

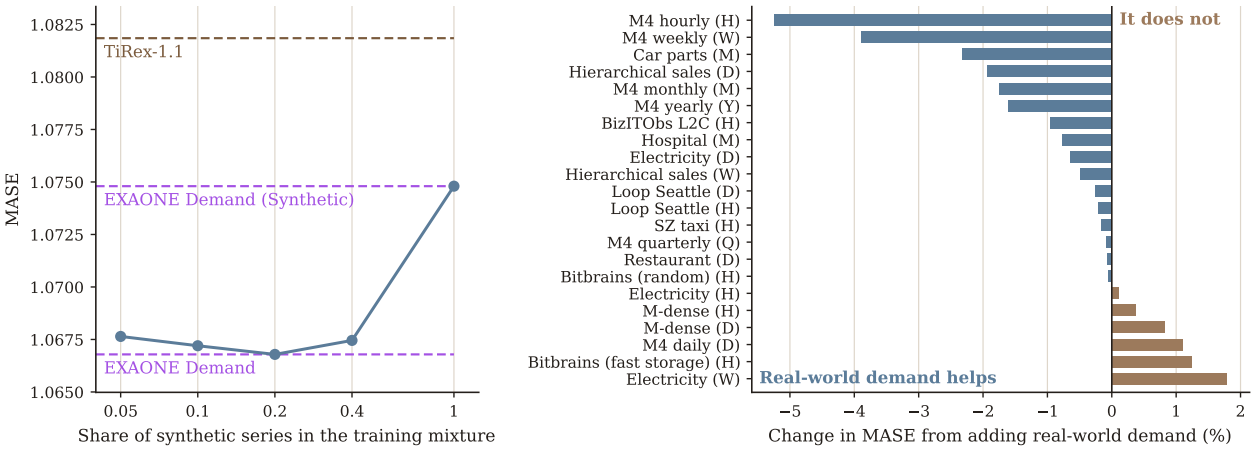


Figure 13: **What the training data contributes.** [Left] The share of synthetic series in the mixture, where each point is the median of three seeds except the synthetic-only point at 1.0, which is the median of its own runs. [Right] The change in MASE from adding real-world demand to the synthetic corpus, on the 22 datasets in the evaluation suite.

models side by side on each of the 22 datasets, where real-world demand lowers the error on 16 of them and raises it on 6, and the datasets it helps most are helped several times as much as the ones it costs anything on are hurt. What the real-world corpus buys is therefore held across the suite rather than produced by a handful of rows.

6 Conclusion

We took a general TSFM as given and asked what a corpus and an adapter built for *demand* are worth. The corpus is assembled from real-world sources, verified against outside counts, separated from the evaluation data at the level of *raw values*, and completed by a synthetic generator. On this corpus, EXAONE Demand trains an MoE adapter whose low-rank branches are mixed by the demand behaviour of each series, and both versions, one trained on real-world and synthetic demand and one on synthetic data alone, outperform every compared TSFM on 22 held-out datasets.

Limitation and future works. The router reads the context window alone, and the events that drive demand, namely promotions, holidays, and price changes, reach the model only through the trace they leave in the series, which makes a covariate-aware router the natural next step. The four classes and the number of routed branches are fixed rather than learned, and stock-outs are modelled only by the synthetic generator, which leaves the recovery of latent demand from censored sales open for the next version of the model, together with a router that is supervised by forecast error.

References

[1] Aslan Ahmedov. Walmart sales forecast. Kaggle dataset, https://www.kaggle.com/datasets/aslanahmedov/walmart-sales-forecast, 2022.

[2] Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. GIFT-Eval: A Benchmark For General Time Series Forecasting Model Evaluation. In *NeurIPS 2024 TSALM Workshop*, 2024.

[3] Abdul Fatir Ansari, Oleksandr Shchur, Jaris Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, Mononito Goswami, Shubham Kapoor, Danielle C. Maddix, Pablo Guerron, Tony Hu, Junming Yin, Nick Erickson, Prateek Mutalik Desai, Hao Wang, Huzefa Rangwala, George Karypis, Yuyang Wang, and Michael Bohlke-Schneider. Chronos-2: From Univariate to Universal Forecasting. https://arxiv.org/abs/2510.15821, 2025.

[4] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the Language of Time Series. *TMLR*, 2024.

[5] Andreas Auer, Patrick Podest, Daniel Klotz, Sebastian Böck, Günter Klambauer, and Sepp Hochreiter. TiRex: Zero-Shot Forecasting Across Long and Short Horizons with Enhanced In-Context Learning. In *NeurIPS*, 2025.

[6] Australian Energy Market Operator. Aggregated price and demand data. https://aemo.com.au/energy-systems/electricity/national-electricity-market-nem/data-nem/aggregated-data, 2026.

[7] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. https://arxiv.org/abs/1803.01271, 2018.

[8] George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, 1970.

[9] Daqing Chen. Online Retail II. UCI Machine Learning Repository, https://doi.org/10.24432/C5CG6D, 2012.

[10] Mouxiang Chen, Lefei Shen, Zhuo Li, Xiaoyun Joy Wang, Jianling Sun, and Chenghao Liu. VisionTS: Visual Masked Autoencoders Are Free-Lunch Zero-Shot Time Series Forecasters. In *ICML*, 2025.

[11] Si-An Chen, Chun-Liang Li, Nate Yoder, Sercan O. Arik, and Tomas Pfister. TSMixer: An All-MLP Architecture for Time Series Forecasting. *TMLR*, 2023.

[12] Citi Bike system data. https://citibikenyc.com/system-data, 2026.

[13] Ben Cohen, Emaad Khwaja, Kan Wang, Charles Masson, Elise Ramé, Youssef Doubli, and Othmane Abou-Amal. Toto: Time Series Optimized Transformer for Observability. https://arxiv.org/abs/2407.07874, 2024.

[14] J. D. Croston. Forecasting and Stock Control for Intermittent Demands. *Operational Research Quarterly*, 23(3):289–303, 1972.

[15] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *ACL*, pages 1280–1297, 2024.

[16] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. Long-term Forecasting with TiDE: Time-series Dense Encoder. *TMLR*, 2023.

[17] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024.

[18] Datadog. BOOM: Benchmark of observability metrics. https://huggingface.co/datasets/Datadog/BOOM, 2025.

[19] Divvy system data. https://divvybikes.com/system-data, 2026.

[20] EINK. CleanTS-65M. Hugging Face model, https://huggingface.co/EINK/CleanTS-65M, 2026.

[21] Vijay Ekambaram, Arindam Jati, Pankaj Dayama, Sumanta Mukherjee, Nam H. Nguyen, Wesley M. Gifford, Chandra Reddy, and Jayant Kalagnanam. Tiny Time Mixers (TTMs): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series. In *NeurIPS*, 2024.

[22] Kun Feng, Shaocheng Lan, Yuchen Fang, Wenchao He, Sihan Lu, Shuqi Gu, Lintao Ma, Xingyu Lu, and Kan Ren. Kairos: Toward Adaptive and Parameter-Efficient Time Series Foundation Models. https://arxiv.org/abs/2509.25826, 2025.

[23] Xinghong Fu, Yanhong Li, Georgios Papaioannou, and Yoon Kim. Reverso: Efficient Time Series Foundation Models for Zero-shot Forecasting. https://arxiv.org/abs/2602.17634, 2026.

[24] Azul Garza, Cristian Challu, and Max Mergenthaler-Canseco. TimeGPT-1. https://arxiv.org/abs/2310.03589, 2023.

[25] Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. Monash Time Series Forecasting Archive. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[26] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. MOMENT: A Family of Open Time-series Foundation Models. In *ICML*, 2024.

[27] Lars Graf, Thomas Ortner, Stanisław Woźniak, and Angeliki Pantazi. FlowState: Sampling-Rate-Equivariant Time-Series Forecasting. In *ICML*, 2026.

[28] Carlos Quesada Granja, Cruz Enrique Borges Hernández, Leire Astigarraga, and Chris Merveille. GoiEner smart meters data. Zenodo, https://doi.org/10.5281/zenodo.7362094, 2022.

[29] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. In *COLM*, 2024.

[30] Tao Hong and Shu Fan. Probabilistic Electric Load Forecasting: A Tutorial Review. *International Journal of Forecasting*, 32(3):914–938, 2016.

[31] Shi Bin Hoo, Samuel Müller, David Salinas, and Frank Hutter. From Tables to Time: Extending TabPFN-v2 to Time Series Forecasting. https://arxiv.org/abs/2501.02945, 2025.

[32] International Monetary Fund. IMF PortWatch. https://portwatch.imf.org/, 2026.

[33] Kaggle. Walmart recruiting – store sales forecasting. https://www.kaggle.com/competitions/walmart-recruiting-store-sales-forecasting, 2014.

[34] Kaggle. Rossmann store sales. https://www.kaggle.com/competitions/rossmann-store-sales, 2015.

[35] Kaggle. Web traffic time series forecasting. https://www.kaggle.com/competitions/web-traffic-time-series-forecasting, 2017.

[36] Kaggle. Corporación favorita grocery sales forecasting. https://www.kaggle.com/competitions/favorita-grocery-sales-forecasting, 2018.

[37] Kaggle. Store item demand forecasting challenge. https://www.kaggle.com/competitions/demand-forecasting-kernels-only, 2018.

[38] Kaggle. Food demand forecasting. Kaggle dataset, https://www.kaggle.com/datasets/kannanaikkal/food-demand-forecasting, 2026.

[39] Seunghan Lee, Juri Hong, Kibok Lee, and Taeyoung Park. Sequential Order-Robust Mamba for Time Series Forecasting. In *NeurIPS 2024 TSALM Workshop*, 2024.

[40] Seunghan Lee, Jaehoon Lee, Jun Seo, Junhyeok Kang, Sangjun Han, Sungdong Yoo, Minjae Kim, Tae Yoon Lim, Dongwan Kang, Hwanil Choi, Soonyoung Lee, and Wonbin Ahn. Beyond Magnitude and Shape: A Direction-Aware Loss for Time Series Forecasting. https://arxiv.org/abs/2608.01857, 2026.

[41] Seunghan Lee, Jaehoon Lee, Jun Seo, Tae Yoon Lim, Dongwan Kang, Hwanil Choi, Minjae Kim, Sungdong Yoo, Junhyeok Kang, Sangjun Han, Soonyoung Lee, and Wonbin Ahn. EXAONE Finance 1.0: An Attention-free Time Series Foundation Model for Financial Time Series. https://arxiv.org/abs/2609.04239, 2026.

[42] Seunghan Lee, Kibok Lee, and Taeyoung Park. ANT: Adaptive Noise Schedule for Time Series Diffusion Models. In *NeurIPS*, 2024.

[43] Seunghan Lee, Taeyoung Park, and Kibok Lee. Learning to Embed Time Series Patches Independently. In *ICLR*, 2024.

[44] Seunghan Lee, Taeyoung Park, and Kibok Lee. Dataset-Driven Channel Masks in Transformers for Multivariate Time Series. In *ICASSP*, 2026.

[45] Xu Liu, Juncheng Liu, Gerald Woo, Taha Aksu, Yuxuan Liang, Roger Zimmermann, Chenghao Liu, Junnan Li, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. Moirai-MoE: Empowering Time Series Foundation Models with Sparse Mixture of Experts. In *ICML*, 2025.

[46] Yong Liu, Guo Qin, Zhiyuan Shi, Zhi Chen, Caiyin Yang, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Sundial: A Family of Highly Capable Time Series Foundation Models. In *ICML*, 2025.

[47] Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative Pre-trained Transformers Are Large Time Series Models. In *ICML*, 2024.

[48] Donghao Luo and Xue Wang. ModernTCN: A Modern Pure Convolution Structure for General Time Series Analysis. In *ICLR*, 2024.

[49] Ernesto Aguilar Madrid. Short-term electricity load forecasting (Panama case study). Mendeley Data, https://doi.org/10.17632/byx7sztj59.1, 2021.

[50] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1):54–74, 2020.

[51] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M5 competition: Background, organization, and implementation. *International Journal of Forecasting*, 38(4):1325–1336, 2022.

[52] Vladyslav Moroshan, Julien Siems, Arber Zela, Timur Carstensen, and Frank Hutter. TempoPFN: Synthetic Pre-training of Linear RNNs for Zero-shot Time Series Forecasting. https://arxiv.org/abs/2510.25502, 2025.

[53] New York City Taxi and Limousine Commission. TLC trip record data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page, 2026.

[54] NHS Business Services Authority. Prescribing and dispensing open data. https://opendata.nhsbsa.net/, 2026.

[55] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *ICLR*, 2023.

[56] NYC Open Data. 311 service requests from 2020 to present. https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2020-to-Present/erm2-nwe9, 2026.

[57] Open Power System Data. https://open-power-system-data.org/, 2020.

[58] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, Marin Biloš, Sahil Garg, Anderson Schneider, Nicolas Chapados, Alexandre Drouin, Valentina Zantedeschi, Yuriy Nevmyvaka, and Irina Rish. Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting. https://arxiv.org/abs/2310.08278, 2023.

[59] Rohit Sahoo. Superstore sales dataset. Kaggle dataset, https://www.kaggle.com/datasets/rohitsahoo/sales-forecasting, 2026.

[60] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.

[61] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *ICLR*, 2017.

[62] Oleksandr Shchur, Abdul Fatir Ansari, Caner Turkmen, Lorenzo Stella, Nick Erickson, Pablo Guerron, Michael Bohlke-Schneider, and Yuyang Wang. fev-bench: A Realistic Benchmark for Time Series Forecasting. `https://arxiv.org/abs/2509.26468`, 2025.

[63] Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. Time-MoE: Billion-Scale Time Series Foundation Models with Mixture of Experts. In *ICLR*, 2025.

[64] Jeremy Stanley. The Instacart online grocery shopping dataset 2017. `https://tech.instacart.com/3-million-instacart-orders-open-sourced-d40d29ead6f2`, 2017.

[65] Aris A. Syntetos and John E. Boylan. On the Bias of Intermittent Demand Estimates. *International Journal of Production Economics*, 71(1-3):457–466, 2001.

[66] Aris A. Syntetos, John E. Boylan, and J. D. Croston. On the categorization of demand patterns. *Journal of the Operational Research Society*, 56(5):495–503, 2005.

[67] Artur Trindade. ElectricityLoadDiagrams20112014. UCI Machine Learning Repository, `https://doi.org/10.24432/C58C86`, 2015.

[68] U.S. Energy Information Administration. Hourly electric grid monitor (form EIA-930). `https://www.eia.gov/electricity/gridmonitor/`, 2026.

[69] VN1 forecasting accuracy challenge. `https://www.datasource.ai/en/home/data-science-competitions-for-startups/phase-2-vn1-forecasting-accuracy-challenge/description`, 2024.

[70] Xue Wang, Tian Zhou, Jinyang Gao, Bolin Ding, and Jingren Zhou. Output Scaling: YingLong-Delayed Chain of Thought in a Large Pretrained Time Series Forecasting Model. `https://arxiv.org/abs/2506.11029`, 2025.

[71] Zihan Wang, Fanheng Kong, Shi Feng, Ming Wang, Xiaocui Yang, Han Zhao, Daling Wang, and Yifei Zhang. Is Mamba Effective for Time Series Forecasting? *Neurocomputing*, 619:129178, 2025.

[72] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified Training of Universal Time Series Forecasting Transformers. In *ICML*, 2024.

[73] Shifeng Xie, Vasilii Feofanov, Jianfeng Zhang, Themis Palpanas, and Ievgen Redko. CauKer: Classification Time Series Foundation Models Can Be Pretrained on Synthetic Data. In *ICLR*, 2026.

[74] Ted Zadouri, Ahmet Üstün, Arash Ahmadian, Beyza Ermiş, Acyr Locatelli, and Sara Hooker. Pushing Mixture of Experts to the Limit: Extremely Parameter Efficient MoE for Instruction Tuning. In *ICLR*, 2024.

[75] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are Transformers Effective for Time Series Forecasting? In *AAAI*, 2023.

[76] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. In *AAAI*, volume 35, pages 11106–11115, 2021.

[77] Zhuohang Zhu, Haodong Chen, Qiang Qu, and Vera Chung. FinCast: A Foundation Model for Financial Time-Series Forecasting. In *CIKM*, pages 4539–4549, 2025.

A Hyperparameters

This appendix records every setting behind the runs of §5, so that the model of §4 can be reproduced from the description alone. Table 10 lists them in four groups, namely the frozen backbone, the adapter of Equation 11, the router and its supervision, and the optimisation. Every value is held fixed across runs, and the data mixture of §5.7 is the one axis that we sweep across the whole study, at four shares of synthetic series and three seeds for each.

Table 10: **Hyperparameters of EXAONE Demand.** The backbone group describes the frozen model that we adapt, and the remaining three groups describe what we add to it and how it is trained on the demand corpus.

Group	Setting	Value
Backbone (frozen)	Blocks	24
	Model width	1,024
	Embedding width	2,048
	Attention heads	16
	Context length	8,192
	Input and output patch	16
	Output patches per step	4
	Quantile levels	21, from 0.01 to 0.99
Adapter	Adapted projections	W_q, W_k, W_v, W_o, and the two feed-forward projections
	Adapted layers	$24 \times 6 = 144$
	Branches E	5, namely 1 shared and 4 routed
	Shared rank r_0	16
	Routed rank $r_1 \dots r_4$	4
	Scale α, dropout	8, 0.05
Router and supervision	Router width h	32
	Router temperature τ	1.0
	Router learning-rate multiplier	100
	Shared weight w	0.5
	Soft cuts τ_a, τ_c	0.35, 0.60
	Sharpening T	0.5
	Auxiliary weight λ	0.1
Optimisation	Steps	9,000
	Learning rate	1.5×10^{-5}
	Weight decay	0.1
	Warm-up steps	300
	Batch size per device	12, accumulated over 4 steps
	Devices and precision	8 GPUs with distributed data parallel (DDP), `bf16` mixed
	Validation interval	Every 1,500 steps

The backbone is loaded from its zero-shot checkpoint, and no weight of it receives a gradient at any point. The router learning-rate multiplier deserves a note of its own, because the router is the one part of the model that has to move far from its initialisation within the same budget as the branches. At the base learning rate it barely leaves the uniform mixture, which produces the collapse described in §4.5 without any other symptom in training.

B Related Work

Demand forecasting. Demand forecasting has a literature of its own, in which intermittent demand is handled by estimators that forecast the size of a non-zero demand and the interval between two of them separately [14, 65], and a series is placed in one of four classes by its average inter-demand interval and by the dispersion of its non-zero values [66]. Classical statistical models fit one estimator per series [8], while forecasting competitions and deep global models fit one model to a collection of related series [51, 60], and energy load forecasting has followed the same path [30]. Our router is supervised by the classical taxonomy, and it carries the separation between classes into a foundation model without giving up the single set of weights that makes one useful in practice.

General time series forecasting models. Task-specific forecasting models are trained on one dataset at a time, where patch-based transformers made long contexts affordable [55, 76], and linear, convolutional, and state space models showed that much of the gain survives without attention [7, 11, 16, 29, 39, 43, 48, 71, 75]. Around these models, prior work has studied what the loss should reward [40] and how to generate series [42], each on one dataset at a time.

Time series foundation models. TSFMs are pretrained on a corpus drawn from many domains and transfer to series that they have never seen [3, 4, 5, 13, 17, 21, 24, 26, 44, 45, 46, 47, 58, 63, 72], and they are ranked by benchmarks assembled in the same way [2, 25]. Mixture-of-experts layers [61] have entered TSFMs as a way to scale capacity [45, 63], whereas we use a mixture of low-rank experts [74] with a shared expert [15] to specialise one frozen backbone by demand behaviour. Synthetic data is the standard answer to a corpus that cannot be collected at the required scale [4, 73], and specialising a general backbone to one domain has been carried furthest in finance [41, 77], where a companion effort to ours adapts the same backbone. The present report is its demand-side counterpart, where the properties that a general corpus under-represents are intermittency, short histories, censoring, and exogenous events.

C Conversion Schema and Verification

This appendix records the schema that every source is converted into, the per-series index that is built alongside it, and the checks that a converted source is required to pass before it is admitted to the corpus, as referred to in §2.2.

All sources are converted to a single schema of `(series_id, timestamp, target)`, and we build a per-series index next to the values themselves, since the checks below compare counts and need every series described first. The index records three kinds of information for every series:

- **Shape.** Length, observation count, zero ratio, and missing ratio.
- **Demand character.** Average inter-demand interval, squared coefficient of variation, and the resulting Syntetos–Boylan class, all computed on the converted series rather than on the raw file as it was published.
- **Provenance.** Source, domain, licence code, commercial-use flag, and split role.

Verification compares the converted counts against numbers established *outside* the pipeline, such as competition documentation, API totals, or the published number of entities, rather than against the conversion log. Four quantities are compared for every source, namely the number of series, the number of observations, the range of timestamps, and the number of distinct entities where the publisher states one. A source is admitted to the corpus once all four agree with the published figures, and the per-series index above is what the first two of them are counted from.

Timestamps are taken from the columns that the source itself carries, and no index is synthesised from an assumed start date. Where a source renames its columns between releases, as NHS prescribing data does in turning `STP_CODE` into `ICB_CODE`, the names are unified before conversion so that every release reaches the same schema.

D Domain Breakdown of the Corpus

Figure 6 shows the corpus by series count, and Table 11 gives the same breakdown numerically while adding the observation count, which tells a different story. The two views disagree because the domains differ in how their series are shaped, and a domain that is large by one of the two measures can be small by the other.

Energy and transport hold *few but very long* series sampled at high frequency, such as meter readings and road sensors. Retail and health hold *many but short* ones, since demand there is recorded per SKU or per practice over a limited window. Energy therefore takes 67.5% of all observations from 39.9% of the series, whereas retail takes 17.2% of the series and only 1.3% of the observations, a gap of fifty times on observations against two on series.

How a domain is assigned. Most sources carry one domain, but the six general-purpose corpora of §2.2 hold many datasets under a single name, and a domain assigned to the corpus would say nothing about the series inside it. The domain is recoverable from the series identifier, which keeps the name of its source dataset, and we map those 153 dataset names onto the same domain vocabulary that the rest of the corpus uses, so that *no series is left without a domain*. The mapping rests on three kinds of evidence: 1) The top-level folders of Time-300B, which are already domain names such as energy, transport, and sales, 2) the per-dataset domain fields recorded when subsets of fev-bench were selected, and 3) the published description of each remaining dataset, read one at a time, which maps `dominick` and `rossmann` to retail, `electricity` to energy, and `azure_vm_traces` to compute. The mapping changes only how series are reported in this appendix, and never which of them are used for training the model.

Table 11: **Corpus composition by domain.** Shares are over the whole converted corpus, including the synthetic portion. Series that arrived inside the general-purpose corpora are counted under the domain of the dataset they came from, not under the corpus that delivered them, so that the shares reflect the data rather than its packaging.

Domain	Series	Share	Observations	Share
Energy	4,507,842	39.87%	32,646,256,620	67.51%
Retail	1,942,682	17.18%	641,490,811	1.33%
Transport	1,578,373	13.96%	9,742,740,721	20.15%
Compute and cloud ops	1,133,431	10.02%	3,686,572,872	7.62%
Web traffic	880,755	7.79%	1,348,375,729	2.79%
Health	817,901	7.23%	32,301,783	0.07%
Economics	372,339	3.29%	100,580,899	0.21%
Synthetic	47,500	0.42%	59,638,451	0.12%
Telecom	20,000	0.18%	90,925,312	0.19%
Tourism	3,576	0.03%	460,340	0.00%
Logistics	2,065	0.02%	5,790,260	0.01%
Public services	276	0.00%	303,449	0.00%
Total	11,306,740	100.00%	48,355,437,247	100.00%